



Передовая инженерная школа
«Средства проектирования и производства электронной
компонентной базы»

Магистерская программа
«Вычислительные системы и ЭКБ»

Дисциплина «Микроконтроллеры и встраиваемые
системы»

Лабораторная работа 02



CMSIS. Семихостинг.

Порты ввода-вывода общего назначения

*Симонов Сергей Борисович
доцент МПСУ МИЭТ, к.т.н.*

Введение

В настоящей работе рассмотрены технологические аспекты программирования встраиваемых систем на примере отладочной платы ST NUCLEO-H745ZI-Q и работой с портами ввода-вывода общего назначения.



Цели работы

- научиться работать с документацией на микроконтроллер;
- освоить применение библиотеки CMSIS;
- изучить основы программирования портов ввода-вывода общего назначения GPIO;
- научиться выполнять сборку, загрузку и тестирование встраиваемых приложений;
- Познакомиться с техникой семихостинга.



Подготовка и защита лабораторной работы

1. Для подготовки к выполнению и защите лабораторной работы необходимо:

- 1) Изучить настоящее пособие и выполнить практические задачи.
- 2) Подготовить конспект и черновые проекты решения практических задач.
- 3) Ознакомиться с рекомендуемой литературой [1],[2],[3],[4].
- 4) Проверить свои знания по перечню вопросов.

2. Защита лабораторной работы

На защиту выносится учебный проект в соответствии с практическим заданием. Созданные программные решения должны компилироваться и запускаться без ошибок и предупреждений.

Во время защиты необходимо:

- продемонстрировать код и работу разработанных программ;
- ответить на теоретические вопросы и вопросы по представленному решению;
- выполнить дополнительное практическое задание по теме работы.



Литература и внешние ресурсы

1. RM0399 - stm32h745.pdf (папка Docs), разделы 12 и 9.7.42
2. UM2408 - nucleo-h745.pdf (папка Docs), раздела 1, 6.1, 6.6, 6.7, 6.9 - 6.11
3. [STM32. Процесс компиляции и сборки прошивки](#)
4. [Don't Repeat Yourself: Как правильно использовать принцип DRY в разработке ПО.](#)
5. [«The GNU linker»](#)
6. [The Red Hat newlib C Library](#)



Теоретический материал

Работа с учебным стендом

Учебный стенд состоит из материнской платы, в которую установлена отладочная плата ST-Nucleo-H745ZI-Q (или аналог). Схема подключения отладочной платы к компьютеру представлена на рис. 1.



Запрещается прикасаться к элементам отладочной платы, за исключением кнопок. Брать и удерживать плату следует только за её края (рёбра). Перед началом работы со стендом необходимо коснуться рукой железного основания стола.

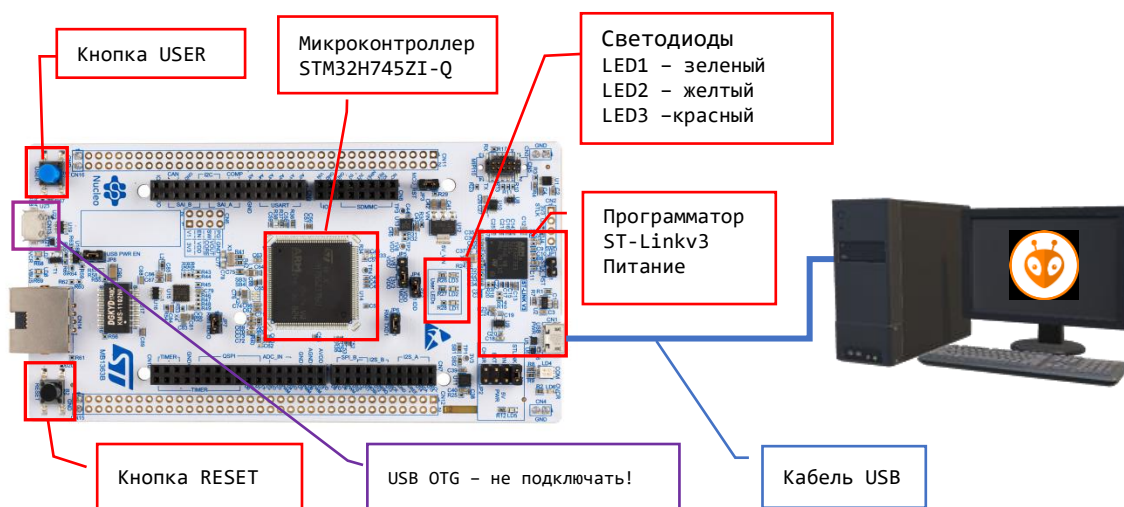


Рисунок 1 – Подключение отладочной платы ST-Nucleo к компьютеру

Документация разработчика

Для создания функционального программного решения встраиваемой системы, разработчику необходимо располагать информацией о компонентах системы и о внешних (сопрягаемых) объектах. Структурно обобщенная модель ВС состоит из следующих ключевых компонент:

- процессор (процессорное ядро);
- внешние по отношению к процессору устройства, расположенные внутри микроконтроллера, такие как SRAM-память, таймер и другие периферийные блоки.
- внешние по отношению к микроконтроллеру устройства, например светодиоды или внешние резонаторы.

В таблице ниже приведены общепринятые названия (виды) документов, к которым следует обращаться для поиска и соответствующей информации.



Основные документы для используемых в лабораторных работах отладочных плат находятся в папке DOCS.

Вид документа	Содержание	Документ (файл) для ST NUCLEO-H745ZI-Q
Program Manual (PM)	Программная модель и периферия процессорного ядра.	PM025-cortexm7.pdf PM0214-cortexm4.pdf
Datasheet (DS)	Спецификация МК: структура; контакты; размеры; допустимые уровни напряжений, токов, температур и т.п.	DS - stm32h745zi.pdf
Reference manual (RM)	Архитектура МК, структура памяти, описание периферийных блоков и регистров МК.	RM0399 - stm32h745.pdf
Application Notes (AN) Technical Notes (TN)	Рекомендации и указания по решению отдельных задач. Детализированное описание.	Например, AN4899 - gpio-and-lowpower.pdf
Application Examples	Демонстрационные программы от производителя микроконтроллера.	https://github.com/ STMicroelectronics/ STM32CubeH7/tree/master/Projects
Errata Sheets (ES)	Документ с описанием аппаратных ошибок в МК и рекомендаций по их обходу.	ES0445-stm32h757xi.pdf
Board User Manual (UM)	Общее описание изделия (отладочной платы) и инструкция по работе с ним	UM2408 - nucleo-h745.pdf
Board Schematic или Board Manual (MB)	Электрическая схема изделия.	MB1363-stm32h745zi.pdf standos_sch.pdf



Файлы с документацией можно найти в материалах к настоящему курсу в папке DOCS.

Стандарт и библиотека CMSIS

CMSIS (Cortex Microcontroller Software Interface Standard) – это стандарт программного интерфейса для микроконтроллеров с процессорным ядром Cortex-M(A), разработанный компанией ARM. Стандарт CMSIS определяет унифицированный программный интерфейс для работы с ядром и периферийными блоками, что обеспечивает: бесшовное объединение ПО из разных источников, упрощение повторного использования кода, переносимость кода между различными микроконтроллерами и средами разработки. Библиотека CMSIS совместима со стандартом MISRA:C.

Компания ARM взаимодействует с производителями микроконтроллеров и инструментальных средств разработки для поддержки CMSIS в их продукции. Как правило, производитель микроконтроллеров с ядром CORTEX-M разрабатывает и предоставляет CMSIS библиотеку, которая учитывает все архитектурные особенности конкретной модели МК.

Центральный компонент библиотеки – CMSIS-CORE – предоставляет механизм доступа к регистрам микроконтроллера и функции для управления прерываниями, системным таймером, памятью и инициализации процессора.

CMSIS содержит и другие полезные компоненты, например:

CMSIS-RTOS –программный интерфейс (API) для взаимодействия с операционными системами реального времени.

CMSIS-DSP – высокоэффективные функции обработки сигналов, такие как фильтрация и преобразование Фурье и SIMD инструкции.

CMSIS-NN – высокоэффективные функции для реализации нейронных сетей.

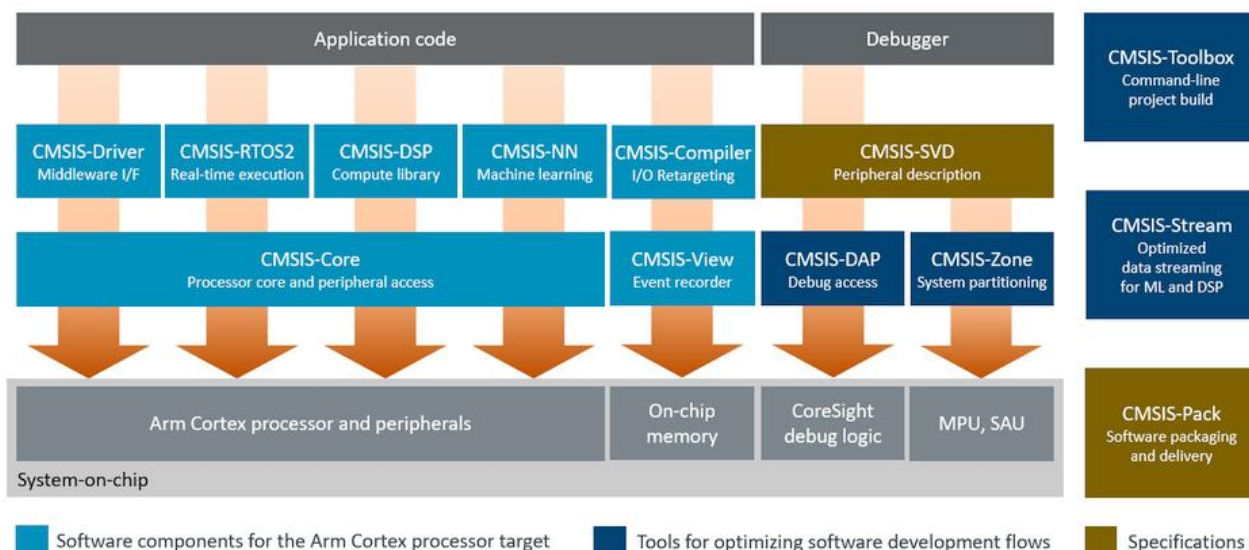


Рисунок 2 – Структура CMSIS

Семихостинг

Важным инструментом для разработки ПО является отладчик, с помощью которого можно отслеживать состояние МК и ход выполнения программы «по шагам». Другим полезным инструментом является трассировка событий, которые передаются в виде сообщений от устройства (микроконтроллера) на хост-компьютер (компьютер разработчика). Трассировка позволяет отслеживать поведение программы не прерывая поток её выполнения.

Семихостинг – это технология, которая даёт возможность использовать в коде программы функции стандартного вывода (например, `printf`) для печати текста непосредственно в консоли отладчика хост-компьютера (рис. 3). При этом каких-либо специальных устройств, за исключением программатора (Debug Probe) не требуется.

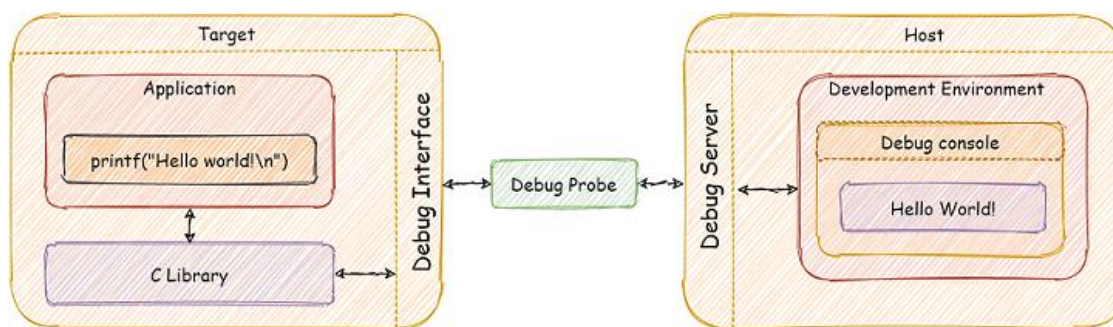


Рисунок 3 – Технология ARM Semihosting

Кроме трассировки событий, семихостинг позволяет делегировать некоторые функции встраиваемой системы, например вывод результатов работы на экран или в файл хост-машины. В частности, таким образом организуется модульное тестирование программы на целевой машине: модульный тест загружается и выполняется на МК, а необходимые для отчета о выполнении теста данные передаются и обрабатываются на хост-компьютере (рис. 4).

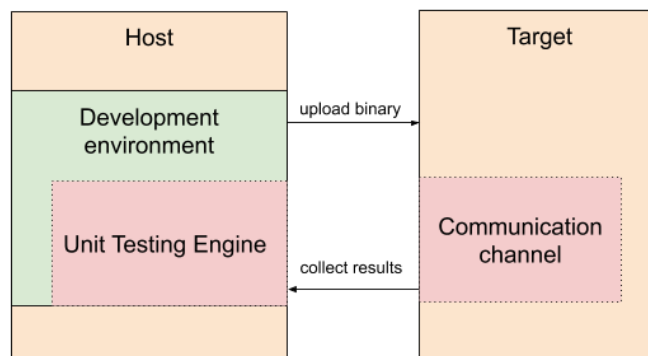


Рисунок 4 – Модульное тестирование встраиваемой системы

Во ряде случаев целесообразно организовать не только вывод, но и ввод сообщений из консоли хост-компьютера в приложение микроконтроллера. Это даёт возможность делегировать хост-компьютеру функции, связанные с подготовкой или выборкой входных данных, например ввод команд оператором.

Существуют несколько технологий организации ввода-вывода между целевой машиной и хост-компьютером:

- ARM Semihosting - передача сообщений в консоль отладчика с помощью программатора;
- Технология Serial Wire View (SWV) – обеспечивает быструю однонаправленную передачу данных по линии SWO интерфейса отладки SWD.
- Ряд коммерческих решений, например Real Time Transfer (RTT), работающие со специализированным программатором компании SEGGER.
- Передача данных через интерфейс UART или USB CDC.

В настоящей работе мы обеспечим ввод и вывод между отладочной платой STM32H745ZI-Q и компьютером с помощью виртуального COM порта, встроенного в программатор ST-Link V3. Этот способ сопряжения не требует дополнительного оборудования или кабелей (рис. 5)

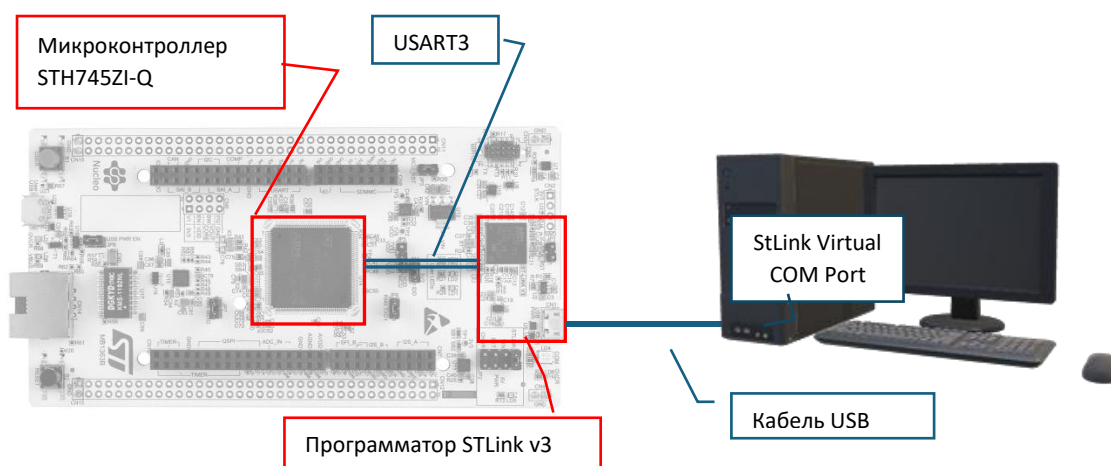


Рисунок 5 – Схема организации ввода-вывода.

Порты ввода-вывода общего назначения (GPIO)

Выводы, или так называемые «пины», микроконтроллера распределены по портам ввода-вывода, которые именуются буквами: GPIOA, GPIOB и т.д. Каждый порт содержит 16 выводов с номерами от 0 до 15. Конкретный вывод принято обозначать записью «PXY» где X – порт, а Y – номер пина. Например, PA2 – пин 2 порта GPIOA.

Каждый вывод может быть сконфигурирован для выдачи логических уровней напряжения (цифровой выход), либо для считывания логического состояния (цифровой вход). Также существует режим «альтернативная функция», которые позволяет связать вывод с определённым периферийными блоком микроконтроллера, например с цифро-аналоговым преобразователем (ЦАП). В этом режиме управление выводом берет на себя соответствующий периферийный блок.

Порты ввода-вывода являются периферийными устройствами МК, регистры которых отображаются в статическую память. Управление работой GPIO осуществляется путём чтения и записи значений в соответствующие регистры.



Ознакомьтесь с разделом «12 General-purpose I/Os (GPIO)» документа RM0399. Заполните в конспекте следующую таблицу для различных регистров GPIO (одна строка уже заполнена для примера).

Регистр	Назначение	Варианты конфигурации для отдельного вывода
GPIO_OTYPER	Конфигурирует тип цифрового вывода	«Push Pull» (=0) или «Open Drain» (=1)
...	...	...



Порядок выполнения работы

Часть 1. Создание проекта CMSIS

1. В среде PlatformIO + VSCode создайте проект для отладочной платы ST Nucleo H745ZI-Q и фреймворка CMSIS. Для этого можно воспользоваться командой

```
pio project init -b nucleo_h745zi_q.
```

После создание проекта может потребоваться активация расширения PlatformIO. Для этого можно использовать команду *Developer: Restart Window*.

2. В состав проекта CMSIS входят три обязательных файла: startup-файл, system-файл и скрипт компоновщика. По умолчанию PlatformIO автоматически включает в сборку указанные файлы из пакета CMSIS для целевого микроконтроллера. Однако эти системные файлы являются неотъемлемой частью кодовой базы, могут модифицироваться и понадобятся нам для работы. Поэтому настроим проект таким образом, чтобы системные файлы хранились в папке `system`.

2.1. В папке проекта создайте папку `system` и скопируйте в неё системные файлы CMSIS для микроконтроллера STM32H745ZI-Q и задайте им файлы простые имена в соответствии с таблицей.



Редактор VsCode позволяет копировать файлы путём перетаскивания мышкой из Проводника во вкладку File Explorer.

Расположение файла	Новое имя файла	Примечание
C:\PioVScPort\platformio\packages\framework-cmsis-stm32h7\Source\Templates\gcc\startup_stm32h745xx.S	startup.s	Startup - файл
C:\PlatformIO\packages\framework-cmsis-stm32h7\Source\Templates\system_stm32h7xx_dualcore_bootcm7_cm4gated.c	system.c	System - файл
C:\PlatformIO\packages\framework-cmsis-stm32h7\Source\Templates\gcc\linker\stm32h745xx_flash_CM7.ld	h745cm7_flash.ld	Скрипт компоновщика (linker script)

2.2. Включите в сборку проекта скопированные системные файлы вместо «автоматического» набора. Для этого в файле проекта `platformio.ini` создайте общее окружение `[env]` и определите в ней специальные параметры. Также добавьте частное окружение для программы мигания светодиодом, которая будет состоять из одного файла `hello_led.c`. (см. листинг 1).

Листинг 1: `platformio.ini`



```
1 [env]
2 platform = ststm32
3 board = nucleo_h745zi_q
4 framework = cmsis
5 build_flags = -std=c11 -Wall -Wextra -D CORE_CM7
6 board_build.ldscript = $PROJECT_DIR/system/h745cm7_flash.ld
7 board_build.cmsis.startup_file = broken_path
8 board_build.cmsis.system_file = broken_path
9
10 [env:hello_led]
11 build_type = debug
12 build_src_filter = +<$PROJECT_DIR/system/*.c>
13                   +<$PROJECT_DIR/system/*.s>
14                   +<*>
```

3. Создайте в папке `src` файл `hello_led.c` и введите текст программы, которая мигает желтым светодиодом отладочной платы. Светодиод подключен к выводу PE1.

Листинг 2: `src/hello_led.c`



```
1 #include <stm32h7xx.h> // основной заголовочный файл CMSIS для МК серии H7
2 #include <myled.h>
3
4 __attribute__((optimize("-O0"))) //директива отключает оптимизацию кода компилятором для
5 этой функции
6 static void delay(int ms) {
7     volatile int counter = SystemCoreClock / 1000 / 6 * ms ;
8     while (counter > 0) counter -= 1;
9 }
10
11 int main() {
12     myled_enable(); // инициализация вывода светодиода
13     while(1){
14         myled_toggle(); // переключение «вкл <-> откл»
15         delay(500); // пауза между переключениями
16     };
17 }
```

4. Используемые в программе функции `myled_enable()` и `myled_toggle()` реализованы в виде библиотеки. Добавьте файлы `myled.h` и `myled.c` в папку `lib/myled`.

Листинг 3: `lib/myled/myled.h`



```
1 #pragma once
2 void myled_enable();
3 void myled_toggle();
4 void myled_disable();
```


Листинг 4: lib/myled/myled.c



```

1  #include "myled.h"
2  #include <stm32h7xx.h>
3
4  void myled_enable() {
5      RCC->AHB4ENR |= RCC_AHB4ENR_GPIOEEN;    // включаем тактирование GPIOE
6      GPIOE->MODER &= ~GPIO_MODER_MODE1_Msk;  // включаем режим "output" ...
7      GPIOE->MODER |= GPIO_MODER_MODE1_0;      // ... для pin 1
8  }
9
10 void myled_toggle() {
11     GPIOE->ODR ^= 2; // ; "исключающее или" с единицей меняет 0->1 и 1->0
12 }
13
14 void myled_disable() {
15     GPIOE->MODER &= ~GPIO_MODER_MODE1_Msk;    // включаем режим "analog" ...
16     GPIOE->MODER |= GPIO_MODER_MODE1_0 | GPIO_MODER_MODE1_1; // ... для pin 1
17 }

```

Часть 2. Анализ программы

1. Откройте и изучите скрипт компоновщика h745cm7_flash.ld.

Скрипт компоновщика содержит правила, по которым компоновщик создает исполняемый ELF-файл.

Язык скрипта компоновщика описан в документации [5]. Скрипт начинается с команды ENTRY, указывающей на функцию, с которой начинается выполнение программы. Эта информация нужна компоновщику для оптимизации размера исполняемой программы – не используемые функции и переменные не будут включены в создаваемый ELF-файл. Обратите внимание, что выполнение программы начинается не с функции main().

Следом идет определение ряда констант и команда MEMORY, в которой определяются доступные области памяти, их размер и атрибуты.

Далее следует большая команда SECTIONS, в которой определяется содержимое секций и их расположение в памяти МК.

После включения питания и/или сброса процессора таблица векторов прерываний находится по адресу ноль. Заметим, что секция с названием. *isr_vector* описана первой и будет размещена в начале FLASH памяти, а значит будет доступна для чтения и выполнения по нулевому адресу. После сброса процессор находит по адресу 0 первый вектор таблицы прерываний для инициализации указателя стека SP, а следующий вектор для определения адреса функции обработчика.

Также во флешь памяти размещена секция *.text*, которая содержит основные инструкции программы (функции в языке C), и секция *.rodata* для хранения константных данных.

Секция *.data* предназначена для хранения инициализированных данных. Обратите внимание на строку «>RAM AT> FLASH», которая сообщает компоновщику, что значение данных будут записаны во флешь-память, но обращение к ним будет происходить по адресам, расположенным в памяти RAM. Ответственность за копирование инициализированных значений из FLASH в RAM до первого обращения к переменным этой секции по имени лежит на программе (выполняется в стартовом коде).

Также в скрипте компоновщика определены специальные секции:

- *ARM.extab* и *ARM.exidx* – секции необходима для реализации бинарного интерфейса обработки программных исключений (ARM EHABI - Exception Handling Application Binary Interface), который необходим для поддержки механизма исключения в языке C++.

- Секции `.preinit_array` и `.init_array` содержат массив указателей на функции, которые должны быть вызваны до и во время инициализации программы, а секция `.fini_array` – во время завершения программы. Этот механизм используется для вызова конструкторов статических объектов C++.
- Секция `.ARM.attributes` используется для хранения метаданных, которые описывают различные атрибуты и характеристики программы или библиотеки. Эти атрибуты содержат информацию о версии компилятора, используемых опциях компиляции, поддерживаемых архитектурных особенностях и т.п. Секция `.ARM.attributes` используется отладчиком или другими инструментами для получения сведений об исполняемом файле.

Директива `/DISCARD/` используется для исключения определенных объектов из ELF фала и содержит библиотеки `libc.a`, `libm.a` и `libgcc.a`. Во встраиваемых системах, вместо этих библиотек используются специальные библиотеки, такие как Newlib или Newlib-nano, которые позволяют адаптировать работу функций стандартной библиотеки.

Удалите из скрипта компоновщика атрибут (READONLY) при определении секций. Это необходимо, поскольку версия компилятора по умолчанию gcc 7.2.1 не поддерживает этот атрибут.



Редактор VsCode позволяет автоматизированно заменить один текст другим.
Нажмите CTRL+F и замените строку (READONLY) на пустую строку.

2. Startup-файл отвечает за первичную конфигурацию микроконтроллера после сброса. Откройте файл `system/startup.s` и изучите стартовый код.



2.1. Найдите секцию `.isr_vector` и ознакомьтесь с вектором прерываний.

2.2. Найдите определение функции `ResetHandler` в секции `.text`. Попробуйте составить обобщенный алгоритм функции `ResetHandler` и запишите его в конспект.



Краткая справка по некоторым директивам языка ассемблер:

`b` или `bx` – безусловный переход на указанный адрес.

`bl` – безусловный переход с возвратом;

`ldr` – загрузка в регистр, например, инструкция `ldr r3, [r3, r1]` загружает значение из памяти по адресу, который получается сложением значений регистров `r3` и `r1`, и сохраняет это значение в регистр `r3`;

`str` – выгрузка из регистра;

`cmp` и `bcc` – сравнение и условный переход, если результат сравнения «больше или равно».

2.3. Найдите в коде реализацию обработчика «по умолчанию» `DefaultHandler`.

3. Откройте файл `system/system.c` и изучите код системного файла.

3.1. Обратите внимание на макросы `HSE_VALUE`, `CSI_VALUE`, `HSI_VALUE`, которые соответствуют частотам внешнего и внутренних генераторов тактовых импульсов МК, а также на макросы `CORE_CM7` и `CORE_CM4`. Значение этих макросов можно установить (переопределить) в параметре `build_flags` файла проекта или изменить непосредственно в файле `system.c`.

Проверьте значение используемой по умолчанию частоты для внешнего генератора HSE, с помощью документа UM2408. При необходимости переопределите макрос, указав правильное значение.

3.2. Работа функции `ExitRun0Mode` также зависит от макросов.

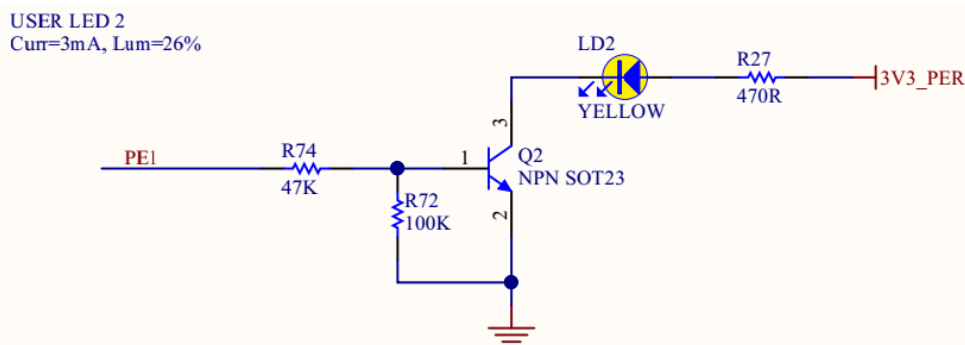
Определите схему питания отладочной платы, которая используется «по умолчанию» (раздел 6.4.8 UM2408), и добавьте нужный соответствующий макрос в сборку проекта.

3.3. Согласно стандарту CMSIS определяется глобальная переменная *SystemCoreClock* для хранения значения частоты системного тактового сигнала и функция *UpdateSystemCoreClock()* для вычисления (обновления) значения этой переменной по состоянию регистров МК.

4. Откройте файл `src/hello_led.c` и изучите код программы.

Обратите внимание на то, что функция *main()* содержит цикл, который никогда не завершается, (суперцикл). Большинство программ микроконтроллера работают до выключения питания (сброса) и содержат такой суперцикл.

Светодиоды отладочной платы подключены к портам с токоограничивающим резистором. Например, желтый светодиод подключен к порту PE1.



Следовательно, для управления светодиодом, необходимо сконфигурировать вывод в режим выхода *Output Push-Pull* и установить выход в высокий (зажечь) или низкий (погасить) логический уровень.

Прежде чем работать с регистрами периферийного блока необходимо включить тактирование этого блока. За тактирование периферийных блоков в МК отвечает специальный блок RCC. Функция *myled_enable()* включает тактирование порта GPIOE.

Ознакомьтесь с разделом 9.7.42 «RCC AHB4 clock register (RCC_AHB4ENR)» документа RM0399. Обратите внимание на назначение битовых полей регистра. Сопоставьте описание регистра RCC_AHB4ENR с кодом функции *myled_enable()*.

5. Ознакомьтесь с основными структурами и макросами, определенными в CMSIS-CORE.

5.1. Изучите определение объектов RCC, GPIOE и соответствующих структур в файле `stm32h745xx.h`. Обратите внимание каким образом в библиотеке CMSIS реализован доступ к регистрам аппаратных блоков (RCC, GPIO).



Для перехода к определению объекта (например, к RCC или файлу директивы include) наведите курсор мыши на его имя и щелкните по нему удерживая клавишу CTRL. Еще один способ – установить текстовый курсор на имя объекта и нажать клавишу F12. Для того, чтобы вернуться к предыдущему месту работы с кодом используете комбинацию клавиш ALT+ВЛЕВО и ALT+ВПРАВО.

5.2. Чтобы использовать драйвер CMSIS достаточно подключить заголовочный файл `stm32h7xx.h`. С включения этого файла начинается программа `hello_led.c`.

Откройте файл `stm32h7xx.h` и ознакомьтесь с определением специальных макросов CMSIS для работы с регистрами: SET_BIT, CLEAR_BIT, READ_BIT, CLEAR_REG, WRITE_REG, READ_REG, MODIFY_REG. Определите назначение остальных макросов.

Макрос POSITION_VAL используется для определения позиции первого установленного бита (единицы) в значении. В макросе выполняет две операции: __RBIT(VAL) - инвертирует порядок битов в значении VAL; __CLZ(VAL) - возвращает количество ведущих нулей в значении VAL.

В каких местах библиотеки *myled* можно применить эти макросы?



5.3. Выясните, какие еще заголовочные файлы подключаются в файле `stm32h7xx.h`. Запомните или запишите в конспект основные макроопределения, от которых зависит конфигурация библиотеки CMSIS.

Часть 3. Сборка, загрузка в МК и отладка программы

1. Выполните сборку программы в режиме подробного вывода.

Код программы преобразуется в объектный файл. Затем создается файл в формате ELF (*Executable and Linkable Format*), а на заключительном этапе файл прошивки (`firmware.bin`), предназначенный для программирования микроконтроллера.



Изучите сообщения, выводимые в терминал в процессе сборки. Проанализируйте какие программы и с какими параметрами вызываются. Запишите название и назначение программ и параметров, с которыми они запускаются. Для поиска информации о программах и ключах используйте ресурсы сети интернет.



2. Выполните загрузку программы в память МК:

2.1. Подключите отладочную плату к компьютеру согласно рис. 1.

2.2. Выполните загрузку (прошивку) бинарного файла программы.

Способ 1: нажать ALT+CTRL+U.

Способ 2: нажать кнопку Upload (→) в Строке состояния.

Способ 3: ввести команду в терминал PIO: `pio run -t upload -e hello_led -v`.

3. После загрузки программы наблюдайте мигание светодиода.

4. Проведите исследование работы программы с помощью отладчика.

4.1. Поставьте точку останова на 13-й строке `hello_led.c` и запустите программу в режиме отладки (F5).

Будет выполнена компиляция версии программы с включенной отладочной информацией, загрузка во флеш память МК и запуск отладчика.

4.2. В окне Peripherals откройте для просмотра состояние регистров блока GPIOE->ODR.

4.3. Проведите анализ изменения регистров GPIO выполняя программу по шагам (F11, F10).

4.4. Приостановите выполнение программы в функции `delay()`. Далее, в окне Disassembly нажмите на строку «Switch to code». Выполняя программу по шагам, определите сколько инструкций занимает один цикл в функции `delay()`. Запишите это число в конспект.

4.5. Завершите работу отладчика.



Часть 4. Библиотека для реализации двунаправленного семихостинга

1. Библиотека *vterm* позволяет выводить и считывать символы из программы, работающей на отладочной плате ST NUCLEO-H745ZI-Q, на терминал хост-компьютера с помощью функций стандартного ввода. Библиотека устроена следующим образом. Функция `vterm_init()` инициализирует блок USART3 микроконтроллера. Также в файле `vterm.c` переопределяются «слабые» функции библиотеки Newlib для записи и чтения таким образом, чтобы передавать и принимать данные из USART3 [6].

2. Перед использованием функций библиотеки *vterm* или функций ввода/вывода стандартной библиотеки необходимо подключить заголовочный файл (`#include <vterm.h>`) вызвать функцию *vterm_init()*, указав скорость передачи данных, например 115 200 бит/с.

3. Добавьте библиотеку *vterm* в проект (см. папку *Resources*). Библиотека состоит из файлов *vterm.h* и *vterm.c*, которые следует разместить в папку *lib/vterm*. Листинги файлов можно найти в приложении или в папке *Resources*.

4. Откройте файл *vterm.h* и ознакомьтесь с функциями, которые предоставляет библиотека.

5. Измените программу *hello_led.c*, чтобы проверить работу библиотеки:

5.1. Подключите заголовочные файлы:

```
#include <stdio.h>
#include <vterm.h>
```

5.2. Добавьте следующие строки в основной цикл программы:

```
static int counter = 0;
printf("\r%s %d %s", "Светодиод был переключен", counter++, "раз(a)");
```

и вызов функции инициализации перед основным циклом:

```
vterm_init(115200);
```

6. Выполните сборку программы. Убедитесь в отсутствии ошибок. Загрузите программу в МК.

7. Для сопряжения с целевым устройством на хост-компьютере можно использовать любую программу-терминал, например *TerraTerm*. В лабораторных работах предлагается использовать встроенный терминал PlatformIO, называемый *Serial Monitor* или просто - монитор. Параметры его работы можно установить в файле проекта *platformio.ini*.

Добавьте в глобальное окружение [env] параметры с указанием скорости приёма данных хост-компьютером:

```
monitor_speed = 115200
test_speed = 115200
```

8. Запустите *Serial Monitor*

Способ 1: нажать на кнопку *Serial Monitor*  в строке состояния.

Способ 2: нажать CTRL+ALT+S

Способ 3: ввести в панель команд (CTRL+SHIFT+P) команду *PlatformIO: Serial Monitor*

Способ 4: ввести команду в терминал PIO: `pio device monitor -e hello_led`

9. После запуска в нижней панели появится новое окно терминала «*Monitor Task*». В терминале должна отображаться строка с увеличивающимся счетчиком.



В один момент времени может работать только один *Serial Monitor*. Повторный запуск монитора приведет к ошибке.

Часть 5. Выполнение юнит тестирования на микроконтроллере

1. Для конфигурации библиотеки *Unity* используются два файла *unity_config.h* и *unity_config.c*, которые необходимо включить в иерархию тестов. Для всех нижестоящих тестов будет выполнена сборка фреймворка *Unity* с этими конфигурационными файлами.

Создайте в папке *test* директорию *target* и поместите в неё файлы конфигурации *Unity*.

Листинг 5: test/target/unity_config.h



```

1  #ifndef UNITY_CONFIG_H
2  #define UNITY_CONFIG_H
3  #ifdef __cplusplus
4  extern "C" {
5  #endif
6
7  void unity_output_start();
8  void unity_output_char(char);
9
10 #define UNITY_OUTPUT_CHAR(a) unity_output_char(a)
11 #define UNITY_OUTPUT_START() unity_output_start()
12
13 // Другие макроопределения, которые можно изменить при необходимости
14 // #define UNITY_OUTPUT_CHAR_HEADER_DECLARATION
15 // #define UNITY_OUTPUT_FLUSH()
16 // #define UNITY_OUTPUT_FLUSH_HEADER_DECLARATION
17 // #define UNITY_OUTPUT_COMPLETE()
18
19 #ifdef __cplusplus
20 }
21 #endif /* extern "C" */
22 #endif /* UNITY_CONFIG_H */

```

Листинг 6: test/target/unity_config.c



```

1  #include "unity_config.h"
2  #include <stdio.h>
3  #include <stm32h7xx.h>
4  #include <vterm.h>
5
6  // Блокирующая задержка с помощью счетчика DWT (Data Watchpoint and Trace unit)
7  static void delay(int ms) {
8      uint32_t cycles = SystemCoreClock / 1000 * ms;
9      DWT->CTRL |= 1;
10     DWT->CYCCNT = 0;
11     while (DWT->CYCCNT < cycles)
12         __asm("nop");
13 }
14
15 void unity_output_start() {
16     vterm_init(115200);
17     delay(1000); // Задержка для подготовки к приёму данных хост-компьютером
18 }
19
20 void unity_output_char(char ch) { putchar(ch); }

```

2. Проанализируйте код конфигурационных файлов.

2.1. Ряд специальных макросов позволяет настроить функцию вывода фреймворка *Unity*. В файле *unity_config.h* определен макрос `UNITY_OUTPUT_START()`, который однократно вызывается перед началом работы фреймворка, и макрос `UNITY_OUTPUT_CHAR(a)`, задача которого вывести один символ сообщения. Как видно из листингов 5 и 6, макросы определяют вызов функций *unity_output_start()* и *unity_output_char()*, которые, в свою очередь, инициализируют библиотеку *vterm* и используют функцию *putchar* стандартной библиотеки для вывода символа.

2.2. В функцию *unity_output_start()* добавлена задержка для того, чтобы среда разработки PlatformIO на хост-компьютере успела запустить терминал для приёма сообщений, после того как программа теста будет загружена в МК и начнет свое выполнение.

2.3. Код функции задержки *delay()* отличается от аналогичной функции в программе *hello_led.c*. Процессорные ядра Cortex-M имеют опциональный блок отладки, называемый *Data Watchpoint*

and Tracing (DWT), который предоставляет точки останова, трассировку данных и системное профилирование для процессора. Регистр CYCCNT этого блока содержит счетчик циклов, выполненных процессором. Использование этого счетчика позволяет не делать предположений о количестве операций, затрачиваемых циклом проверки.

3. Создайте тест для функции смены состояния светодиода *myled_toggle()*. Для этого создайте файл теста, вставьте и в него код приведенного ниже листинга. Выполните анализ вставленного кода тестовой программы

Листинг 7: test/target/test_myled/myled_toggle.c



```

1  #include <stm32h7xx.h>
2  #include <myled.h>
3  #include <unity.h>
4
5  void setUp() {
6      myled_enable();
7  }
8
9  void tearDown() {
10     myled_disable();
11 }
12
13 void test_myled_toggle() {
14     uint32_t state[3] = {0};
15     state[0] = GPIOE->ODR & GPIO_ODR_OD1;
16     myled_toggle();
17     state[1] = GPIOE->ODR & GPIO_ODR_OD1;
18     TEST_ASSERT_NOT_EQUAL_UINT32_MESSAGE(state[0], state[1],
19                                           "led state not changed after toggle once");
20     myled_toggle();
21     state[2] = GPIOE->ODR & GPIO_ODR_OD1;
22     TEST_ASSERT_EQUAL_UINT32_MESSAGE(state[0], state[2],
23                                       "led state changed after toggle twice");
24 }
25
26 int main() {
27     UNITY_BEGIN();
28     RUN_TEST(test_myled_toggle);
29     return UNITY_END();
30 }
```

4. Перед запуском теста:

4.1. добавьте в файл проекта параметр:

```
test_framework = custom
```

4.2. Поместите файл *test_custom_runner.py* в папку *test*

4.3. Скопируйте файлы *startup.s* и *system.c* в папку *test/target*

5. Запустите тест и убедитесь в получении отчета об успешном прохождении теста.

Практическое задание №1

Напишите макрос *TOGGLE_BIT(REG, BIT)*, который меняет значение бита в регистре по маске. Измените код функций в файле *myled.c*, применив созданный макрос и макросы библиотеки CMSIS для работы с регистрами.

Практическое задание №2

Определите к каким входам подключены красный и зеленый светодиоды отладочной платы (документация находится в папке DOCS).

Библиотека для управления красным/желтым/зеленым светодиодом должна иметь следующий программный интерфейс:

```
typedef enum {  
    led_green = 1,  
    led_yellow = 2,  
    led_red = 4,  
    led_all = 1|2|4  
} led_t;  
  
void led_enable(led_t led); // включает тактирование GPIO блока  
void led_toggle(led_t led);  
void led_on(led_t led);  
void led_off(led_t led);  
void led_disable(led_t led); // переводит вывод в режим Analog
```

Задание выполняется парами.

Первый студент самостоятельно реализует функции библиотеку на языке C или C++.

Второй студент самостоятельно разрабатывает юнит-тесты для библиотеки. Тесты должны проверять функции включения, выключения и переключения для светодиодов всех цветов.

Совместно необходимо добиться прохождения всех созданных тестов над разработанной библиотекой.

*Дополнительные баллы

Дополнительные ставятся за выполнения доп. задания во время защиты.

Также дополнительные баллы предоставляются за «качество» кода, в том числе за следование принципам DRY и KISS в программном решении.



DRY – это принцип разработки программного обеспечения, призванный минимизировать дублирование информации в коде. Согласно DRY, «Каждая единица знания должна иметь единственное, непротиворечивое и авторитетное представление в рамках системы». На практике это значит, что повторяющиеся части кода должны объединяться в общие функции или модули.

Принцип KISS («*Keep It Simple, Stupid*» или «*Keep It Short and Simple*») в разработке ПО определяет «простоту» как одной из ключевых целей и требует избегать использование более сложных средств, чем необходимо.

Для решаемой задачи одним из критериев может являться количество изменений, которое потребуется внести в код при добавлении поддержки библиотекой еще одного светодиода или еще одной функции.



Вопросы для самостоятельной проверки

1. С какой функции начинается выполнение программы в МК после сброса? Какое первое действие этой функции?
2. Какая программная утилита создает бинарный файл прошивки (конспект) ?

3. Какие входные параметры используются для создания ELF-файла (конспект)?
4. Что такое ELF файл и какая информация в нем содержится?
5. Верны ли данные статистики по проценту используемой памяти, выводимые по завершению сборки?
6. В какой секции содержатся данные о версии компилятора и флагах компиляции. Записывается ли эта информация в МК?
7. Что будет если не выполнять копирование секции .data в секцию .ram в startup файле?
8. Какие действия выполняет функция *SystemInit()*?
9. Для чего нужна функция *SystemCoreClockUpdate()*?
10. Для чего нужна переменная *SystemCoreClock*?
11. С помощью каких макроопределений выполняется конфигурация библиотеки CMSIS (конспект)?
12. Какие компоненты имеются в CMSIS.
13. Какие регистры GPIO вы знаете и для чего они нужны (конспект).
14. Что такое семихостинг? Какие технологии семихостинга существуют?
15. Для чего нужна библиотека *Newlib*?
16. Как работает библиотека *vterm*?
17. Для его нужна команда MEMORY в скрипте компоновщика?
18. Какая информация содержится в Reference Manual микроконтроллера?
19. Какая информация содержится в Data Sheet микроконтроллера?
20. Какая информация содержится в Reference Manual?
21. Каким образом настраивается Unity для семихостинга?
22. Что находится в системном (system), а что в startup файле?
23. Каким образом создать программу для ядра CM4?
24. В чем отличие установки значения выхода пина с помощью BSRR и ODR?
25. В каких состояниях может находиться вывод GPIO?
26. На какой частоте работает внешний резонатор HSE (выполнение) ?
27. Для чего нужен блок RCC?
28. В чем принципиальное отличие функций *delay()* в *hello_led.c* и *unity_config.c*?
29. Что такое DRY и KISS?



30. Какие секции определены в скрипте компоновщика и для чего нужна каждая из них?



Приложение 1. Библиотека Vterm

Библиотека *vterm* позволяет выводить и считывать символы из программы NucleoH745 на терминал хост-компьютера с помощью функций стандартного ввода. Состоит из двух файлов, которые следует разместить в папку `lib/term` проекта. Перед использованием функций ввода/вывода необходимо вызвать `vterm_init()`.

Файлы библиотеки находятся в папке Resources.